Terminator: A Custom Terminal

Computing Lab (CS69201) · Term Project
Deadline: November 3, 2026, 11:59 PM

---

1. Overview

Build **Terminator**, a terminal emulator and shell in C for Linux. It draws its own X11 window with multiple tabs and runs commands with pipes, redirection, job control, history, auto-completion and bash scripts. The project exercises the system calls covered in the course (`fork`, `execvp`, `wait`, `pipe`, `dup2`, `open`/`read`/`write`, `select`/`poll`, `signal`/`kill`), together with POSIX threads, semaphores and shared memory.

In the examples, `user@terminator>` is the prompt, `>` is the continuation prompt, and other lines are output. Anything this document does not specify is your design choice; explain it in the design document.

2. Execution modes

Each tab is an independent shell. How tabs run depends on how Terminator is launched:

```
./terminator                # thread mode (default)
./terminator useProcess=True    # process mode
```

In both modes, the commands a user types run as child processes (`fork()` + `execvp()`).

Thread mode (default)

- One process; each tab is a thread. The tabs share the process's memory. Protect shared state such as history with a mutex or semaphore.

Process mode (`useProcess=True`)

- The GUI runs in the parent; each new tab `fork()`s its own process and talks to the GUI through pipes.
- History stays shared: keep it in shared memory, protected by a process-shared semaphore.
- Closing or crashing one tab must not affect the others.

Both modes must behave the same. A command typed in one tab appears in another tab's history; for example `cd` in one tab does not change another tab's directory; Ctrl+C in one tab does not touch another tab's jobs. How the GUI and tabs communicate, and how shared data is laid out, is up to you. In both modes the GUI is the only part that touches X11, and history lives in one shared, locked place.

3. Task 1 — X11 GUI and tabs

Render the terminal in an X11 window with Xlib (`XOpenDisplay`, `XCreateSimpleWindow`, `XNextEvent`, `XDrawString`). It should behave like a bash terminal: prompt, typed input, command output and scrolling. All input and output go through the window, not the console.

- Keep a text buffer per tab and redraw the window from it.
- Turn `KeyPress` events into characters (`XLookupString`).
- Capture command output through pipes and draw it in the tab.
- Tabs can be opened, closed and switched (by keyboard and/or mouse). A tab keeps running while another is shown.
- The event loop must not busy-wait.

```
[ 1: ~/Downloads ] [ 2: /tmp ] [ + ]
user@terminator> ls
abc.txt  def.txt  myprog.c
```

4. Task 2 — Built-in and external commands

External commands run in a child with `fork()` + `execvp()`, and the shell waits for them. Commands that change the shell's own state must be built-ins: at least `cd`, `exit`, `history`, `multiWatch`, `jobs`/`fg`/`bg` and `source`. An unknown command prints an error and the tab stays usable. Keep the last exit status in `$?`.

```
user@terminator> cd ~/Downloads/mydir
user@terminator> gcc -o myprog myprog.c
user@terminator> ./myprog
user@terminator> notacommand
terminator: notacommand: command not found
```

5. Task 3 — Multiline input and quoting

If a command is incomplete when Enter is pressed, show the `>` prompt and keep reading. Support:

- line continuation with a trailing `\`;
- single and double quotes that span several lines;
- command substitution with `$(...)` and backticks, including nested and multi-line ones (innermost runs first);
- bash quoting rules: single quotes are literal, double quotes still expand `$VAR` and substitutions.

```
user@terminator> echo "Isn't process creation exciting?\
> Fork, exec, repeat!"
Isn't process creation exciting?
Fork, exec, repeat!
user@terminator> echo "Files in $(basename $(pwd)): $(ls | \
> wc -l)"
Files in mydir: 3
user@terminator> echo 'literal: $HOME' "expanded: $HOME"
literal: $HOME expanded: /home/user
```

6. Task 4 — I/O redirection

Support `<`, `>` and both together, using `dup()`/`dup2()` in the child. Report a missing input file instead of running the command. Background jobs that read the same file at the same time must not interfere with each other.

```
user@terminator> sort < somefile.txt
user@terminator> ls > abc
user@terminator> ./a.out < infile.txt > outfile.txt
```

7. Task 5 — Pipes

Support pipelines of any length with `pipe()`: N commands need N−1 pipes and N child processes. Pipes can be combined with redirection.

```
user@terminator> ls *.txt | wc -l
user@terminator> cat abc.c | sort | head -5
user@terminator> ls *.txt | xargs rm
```

8. Task 6 — `multiWatch`

`multiWatch ["cmd1", "cmd2", ...]` runs all the commands in parallel, one process each, and prints each command's output as it arrives until the user presses Ctrl+C. (This differs from `watch "cmd1 && cmd2"`, which runs them one after another.)

- Use one pipe per command and wait on all read ends with `select()` or `poll()`; no busy-waiting.
- Like `watch`, re-run a command after it finishes (every 2 s by default).
- Ctrl+C, detected in the X11 event loop, must terminate every child (e.g. with `kill()`); the shell keeps running.

Each piece of output is printed in this format (marks are deducted otherwise):

```
"cmd1", <unix_timestamp>:
-----------------------------------------------------
<output of cmd1>
-----------------------------------------------------
```

```
user@terminator> multiWatch ["date +%T", "ls | wc -l"]
"date +%T", 1790860020:
-----------------------------------------------------
18:37:00
-----------------------------------------------------
"ls | wc -l", 1790860020:
-----------------------------------------------------
3
-----------------------------------------------------
^C
user@terminator>
```

9. Task 7 — Hotkeys

Support the bash-style editing keys below while the user types a command. Detect them in the KeyPress handler (check ControlMask / ShiftMask), update the input buffer and cursor, then redraw.

The keys in the table are defaults. Users must be able to change them, for example through a config file such as ~/.terminatorrc, a built-in command, or a settings prompt. You choose the mechanism. Custom bindings take effect without recompiling, and are still in place after Terminator restarts. If a binding is invalid or conflicts with another one, report it and fall back to the default.

Key	Action
← / →	Move within the line
Ctrl+A / Ctrl+E	Move to the start / end of the line
↑ / ↓	Browse command history
Ctrl+U / Ctrl+K	Delete from the cursor to the start / end of the line
Ctrl+W	Delete the previous word
Ctrl+L	Clear the screen
Ctrl+D	Exit the shell (close the tab) / send EOF
Ctrl+Shift+C / Ctrl+Shift+V	Copy / paste, using X11 selections

10. Task 8 — Signals and job control

- **Ctrl+C** stops the running command and the prompt returns. Terminator itself must never stop.
- **Ctrl+Z** suspends the running command and the prompt returns.
- Support `cmd &` to start a job in the background, and `jobs`, `fg`, `bg` to manage jobs.

Keys arrive as X11 events, so the shell must deliver the signals itself with `kill()`.

```
user@terminator> sleep 100
^Z
[1]+  Stopped                 sleep 100
user@terminator> bg %1
[1]+ sleep 100 &
user@terminator> fg %1
sleep 100
^C
user@terminator>
```

11. Task 9 — Searchable history

- Keep the last 10,000 commands in a history file (as bash does with `~/.bash_history`), shared by all tabs.
- `History command` shows the most recent 1,000 commands.
- **Ctrl+R** shows `Enter search term:` . On Enter, print:
    a. the most recent command that exactly matches the term; otherwise
    b. the command(s) sharing the longest common substring with the term, if that match is longer than 2 characters; otherwise
    c. `No match for search term in history`.

```
user@terminator> (Ctrl+R)
Enter search term: myprog
./myprog < in.txt
gcc -o myprog myprog.c
user@terminator> (Ctrl+R)
Enter search term: xyz
No match for search term in history
```

12. Task 10 — File-name auto-completion

Pressing **Tab** completes the word before the cursor using the files in the tab's current directory. The rest of the line stays as typed, even if the command does not exist.

- One match: complete it.
- Several matches: extend to their longest common prefix; if still ambiguous, show a numbered list and let the user pick.
- No match: do nothing.

With `abc.txt`, `abcd.txt` and `def.txt` in the directory:

```
user@terminator> ./myprog de          ← Tab
user@terminator> ./myprog def.txt
user@terminator> ./myprog def.txt abc   ← Tab
1. abc.txt   2. abcd.txt              ← press 1
user@terminator> ./myprog def.txt abc.txt
```

13. Task 11 — Running bash scripts

Terminator must run bash script files (`./script.sh`, `bash script.sh`, `source script.sh`) by interpreting them itself, not by handing them to `/bin/bash`. Scripts may contain:

- comments, multiple commands, pipes, redirection and command substitution;
- variables (`NAME=value`, `$NAME`), arguments (`$1`, `$2`, …, `$#`) and `$?`;
- `if` / `elif` / `else` / `fi`, possibly nested, with `[ ... ]` tests (e.g. `-eq`, `-gt`, `=`, `-f`, `-d`);
- `exit [n]`.

Loops and functions are not required. A syntax error (such as a missing `fi`) is reported instead of running the script.

```
user@terminator> cat check.sh
#!/bin/bash
if [ -f "$1" ]; then
   echo "File exists, compiling..."
   gcc "$1" -o myprog
else
   echo "Source file missing!"
fi
user@terminator> ./check.sh myprog.c
File exists, compiling...
user@terminator> bash check.sh nothere.c
Source file missing!
```

---

14. Deliverables

Submit your GitHub repository link through the form that will be posted on Moodle. The repository must contain:

1. Complete source code for Terminator.
2. A Makefile. Running `make` must build `./terminator` without errors, using the required X11 compiler flags.
3. A README that explains:what the project is;the dependencies and how to install them (e.g. `libx11-dev`); how to build and run Terminator in both modes (`./terminator` and `./terminator useProcess=True`); which features are implemented and any known limitations and any other information that you feel necessary

4. A design document covering: the X11 GUI and event loop; thread mode vs process mode, and how shared state is synchronised;process and job management;IPC (pipes, `dup2`, redirection);the data structures used for history and auto-completion;signal handling; any other design decisions.

5. Anything else you think helps, such as test scripts, sample config files, demo commands or screenshots.